

BigVGAN 24kHz-to-44kHz Conversion Script

Below is a PyTorch script to migrate the IndexTTS BigVGAN generator from 24 kHz to 44.1 kHz. We adjust the model's sample rate, hop size, and upsampling layers to match NVIDIA's 44 kHz BigVGAN configuration (512x upsampling) ¹ ² while preserving the IndexTTS-specific enhancements (e.g. speaker conditioning). The script loads the original 24 kHz model weights, builds a new 44 kHz model, transfers matching weights, initializes new parameters for changed layers, validates the model structure, and saves the converted weights.

1. Set Up and Configuration Changes

```
import torch
import torch.nn as nn

# Configuration: original 24k vs new 44.1k (NV BigVGAN)
orig_sample_rate = 24000
orig_hop = 256
orig_upsample_rates = [4, 4, 4, 4, 2, 2] # original 24k upsampling 3
orig_upsample_kernels = [8, 8, 4, 4, 4, 4] # original kernel sizes
new_sample_rate = 44100
new_hop = 512
new_upsample_rates = [8, 4, 2, 2, 2, 2] # target 44.1k upsampling 1
new_upsample_kernels = [16, 8, 4, 4, 4, 4] # target kernel sizes
```

Here we specify the new sample rate (44,100 Hz) and hop size (512) to match NVIDIA's 44 kHz BigVGAN ². The upsampling factors are changed from [4,4,4,4,2,2] (product=1024) to [8,4,2,2,2,2] (product=512) and corresponding kernel sizes from [8,8,4,4,4,4] to [16,8,4,4,4,4], following the 44 kHz config ¹. These changes halve the overall upsampling ratio (1024→512) and double the hop length (256→512) so that mel-spectrogram frames cover the same time span at 44.1 kHz as before ⁴.

2. Define Model Architecture with 44.1 kHz Parameters

We define the BigVGAN generator architecture, including the anti-aliased periodic activation (SnakeBeta) and the speaker conditioning modules. The structure is based on IndexTTS BigVGAN with necessary modifications for the new sample rate (e.g. an extra upsampling stage).

```
# Utility: attribute dict for config
class AttrDict(dict):
    """Helper to access dict keys as attributes (for config)."""
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.__dict__ = self

# Periodic SnakeBeta activation (for anti-aliased activation)
```

```

class SnakeBeta(nn.Module):
    def __init__(self, channels, alpha=1.0, alpha_trainable=True, alpha_logscale=False):
        super().__init__()
        # Two trainable parameters per channel: alpha (frequency) and beta (magnitude)
        self.alpha_logscale = alpha_logscale
        if alpha_logscale:
            self.alpha = nn.Parameter(torch.zeros(channels))
            self.beta = nn.Parameter(torch.zeros(channels))
        else:
            self.alpha = nn.Parameter(torch.ones(channels) * alpha)
            self.beta = nn.Parameter(torch.ones(channels) * alpha)
        self.alpha.requires_grad = alpha_trainable
        self.beta.requires_grad = alpha_trainable
        self.eps = 1e-9

    def forward(self, x):
        # SnakeBeta activation:  $x + 1/\beta * \sin^2(\alpha * x)$  5
        alpha = self.alpha.unsqueeze(0).unsqueeze(-1)
        beta = self.beta.unsqueeze(0).unsqueeze(-1)
        if self.alpha_logscale:
            alpha = torch.exp(alpha)
            beta = torch.exp(beta)
        return x + (1.0 / (beta + self.eps)) * torch.sin(alpha * x).pow(2)

# Simple wrapper to integrate anti-aliasing (if any) with activation
class Activation1d(nn.Module):
    def __init__(self, activation_module):
        super().__init__()
        self.activation = activation_module # e.g., SnakeBeta instance
    def forward(self, x):
        return self.activation(x)

```

IndexTTS BigVGAN Generator Definition: We now define the BigVGAN generator class with integrated speaker conditioning. It includes: - **Upsampling layers:** 6 transposed convolutions with new factors `[8,4,2,2,2,2]` and kernels `[16,8,4,4,4,4]` (matching 44.1 kHz config). - **ResBlocks:** Anti-aliased residual blocks (AMPBlock1) with SnakeBeta activation (from IndexTTS). These use the same kernel/dilation pattern as original (e.g. $3 \times 1D$ convs with dilations 1,3,5) ⁶ ⁷. - **Speaker encoder:** an ECAPA-TDNN-based module to produce a 512-dim speaker embedding from reference mel (100 mel bins) ⁸ ⁹. This embedding is added via a 1×1 conv to the generator's conv layers (globally and at each upsampling stage). - **SnakeBeta activations:** used in residual blocks and a final post-net activation, following IndexTTS config (activation `"snakebeta"`, `snake_logscale=True`).

For brevity, we define a minimal ECAPA-TDNN speaker encoder structure sufficient to load the pretrained weights. (In practice, you could import the IndexTTS speaker encoder.) Then we assemble the BigVGAN model:

```

# Minimal ECAPA-TDNN speaker encoder (for weight loading)
class ECAPA_TDNN(nn.Module):
    def __init__(self, mel_bins=100, embed_dim=512):
        super().__init__()

```

```

        self.conv_front = nn.Conv1d(mel_bins, 512, kernel_size=5, dilation=1, padding=2) #
TDNNBlock initial layer
        # Three SE-Res2Net blocks (dilations 2,3,4) with 512 channels each
        self.res2blocks = nn.ModuleList([
            nn.Conv1d(512, 512, kernel_size=3, dilation=2, padding=2), # Res2Net conv (scale
fusion internally)
            nn.Conv1d(512, 512, kernel_size=3, dilation=3, padding=3),
            nn.Conv1d(512, 512, kernel_size=3, dilation=4, padding=4)
        ])
        self.se_blocks = nn.ModuleList([
            nn.Sequential(nn.Conv1d(512, 128, 1), nn.ReLU(inplace=True),
                nn.Conv1d(128, 512, 1), nn.Sigmoid()) # SE-block for each Res2Net
output
        for _ in range(3)
        ])
        self.conv_aggreg = nn.Conv1d(512 * 3, 1536, kernel_size=1) # concatenate 3 skips ->
1536
        self.asp_pool = nn.Sequential(
            nn.Conv1d(1536, 128, kernel_size=1), nn.Tanh(), # attentive stats pooling
            nn.Conv1d(128, 1536, kernel_size=1)
        )
        self.bn = nn.BatchNorm1d(1536 * 2) # BN after pooling
        self.fc = nn.Conv1d(1536 * 2, embed_dim, kernel_size=1) # final linear

    def forward(self, mel, lengths=None):
        x = mel.transpose(1, 2) # (B, mel_bins, T)
        x = nn.functional.relu(self.conv_front(x))
        residuals = []
        out = x
        for conv, se in zip(self.res2blocks, self.se_blocks):
            # Res2Net-style block: conv + SE + residual
            z = conv(out)
            z = se(z) * z # apply SE scaling
            out = nn.functional.relu(z + out) # residual connection
            residuals.append(out)
        # Aggregate multi-layer features
        multi_feat = torch.cat(residuals[1:], dim=1) # skip first layer in concat
        feat = nn.functional.relu(self.conv_aggreg(multi_feat))
        # Attentive statistical pooling (mean+std)
        attn = self.asp_pool(feat) # attention logits per channel
        attn = torch.softmax(attn, dim=2) # normalize across time
        mean = (attn * feat).sum(dim=2, keepdim=True)
        std = torch.sqrt((attn * (feat - mean)**2).sum(dim=2, keepdim=True) + 1e-12)
        pooled = torch.cat([mean, std], dim=1)
        out = self.bn(pooled)
        out = self.fc(out) # (B, embed_dim, 1)
        return out.transpose(1, 2) # (B, 1, embed_dim)

```

BigVGAN Generator Class (44.1 kHz):

```

class BigVGAN_Generator(nn.Module):
    def __init__(self, config):
        super().__init__()
        h = config
        # Pre-convolution to expand GPT acoustic output to initial channels
        self.conv_pre = nn.utils.weight_norm(nn.Conv1d(h.gpt_dim, h.upsample_initial_channel,
kernel_size=7, padding=3))
        # Upsampling layers (transposed convs)
        self.ups = nn.ModuleList()
        in_ch = h.upsample_initial_channel
        for u, k in zip(h.upsample_rates, h.upsample_kernel_sizes):
            out_ch = in_ch // 2 # halve channels at each stage
            convt = nn.utils.weight_norm(nn.ConvTranspose1d(in_ch, out_ch, kernel_size=k,
stride=u, padding=(k - u) // 2))
            self.ups.append(nn.ModuleList([convt]))
            in_ch = out_ch
        # Define residual blocks for each upsampling layer
        self.resblocks = nn.ModuleList()
        for i, _ in enumerate(self.ups):
            ch = h.upsample_initial_channel // (2 ** (i + 1))
            for (k, d) in zip(h.resblock_kernel_sizes, h.resblock_dilation_sizes):
                # Using AMPBlock1 (anti-aliased multi-period resblock) as in IndexTTS BigVGAN 10
                resblock = AMPBlock1(h, channels=ch, kernel_size=k, dilation=d,
activation=h.activation)
                self.resblocks.append(resblock)
            # Post-convolution: SnakeBeta activation + final output conv
            final_ch = h.upsample_initial_channel // (2 ** len(h.upsample_rates))
            if h.activation == "snakebeta":
                self.activation_post = Activation1d(SnakeBeta(final_ch,
alpha_logscale=h.snake_logscale))
            elif h.activation == "snake":
                self.activation_post = Activation1d(SnakeBeta(final_ch,
alpha_logscale=h.snake_logscale)) # (could use Snake if needed)
            else:
                raise ValueError("Unsupported activation type")
            self.conv_post = nn.utils.weight_norm(nn.Conv1d(final_ch, 1, kernel_size=7,
padding=3))
            # Speaker encoder (pre-trained) and conditioning layers
            self.speaker_encoder = ECAPA_TDNN(mel_bins=h.num_mels,
embed_dim=h.speaker_embedding_dim)
            self.cond_layer = nn.Conv1d(h.speaker_embedding_dim, h.upsample_initial_channel,
kernel_size=1)
            if h.cond_d_vector_in_each_upsampling_layer:
                self.conds = nn.ModuleList([
                    nn.Conv1d(h.speaker_embedding_dim, h.upsample_initial_channel // (2 ** (i +
1)), kernel_size=1)
                    for i in range(len(self.ups))
                ])

```

```

def forward(self, x_code, mel_ref):
    """
    x_code: (B, T_code, gpt_dim) acoustic model (GPT) output features
    mel_ref: (B, T_mel, num_mels) reference mel-spectrogram for speaker embedding
    """

    # Compute speaker embedding (d-vector) from reference mel
    d_vec = self.speaker_encoder(mel_ref)          # (B, 1, embed_dim)
    d_vec = d_vec.transpose(1, 2)                  # (B, embed_dim, 1)
    # Expand code features and add global speaker conditioning
    x = x_code.transpose(1, 2)                     # (B, gpt_dim, T_code)
    x = self.conv_pre(x)                           # (B, initial_ch, T_code)
    x = x + self.cond_layer(d_vec)                 # global conditioning
    # Upsampling with per-layer conditioning
    resblock_idx = 0
    for i, up_layers in enumerate(self.ups):
        # Apply transposed conv upsampler(s)
        for convt in up_layers:
            x = convt(x)
        if hasattr(self, "conds"):
            x = x + self.conds[i](d_vec)            # speaker cond at each stage
        # Residual blocks (AMPBlock1) for this upsampling stage
        residual = 0
        for j in range(len(h.resblock_kernel_sizes)):
            residual += self.resblocks[resblock_idx](x)
            resblock_idx += 1
        x = residual / len(h.resblock_kernel_sizes)
    # Post-processing
    x = self.activation_post(x)
    x = self.conv_post(x)
    return torch.tanh(x)

```

(For completeness, we assume `AMPBlock1` is defined similarly to the original BigVGAN's `ResBlock`, applying three `Conv1d` layers with dilations and `SnakeBeta` activations per block ¹² ¹³. The details of `AMPBlock1` are omitted here for brevity, as the weights will be loaded from the pretrained model.)

3. Load Pretrained Weights and Initialize New Model

We load the original BigVGAN generator weights (`bigvgan_generator.pth`) and create a new model instance with the 44.1 kHz config. Any layers that differ in shape (e.g. first upsampling conv) will be randomly initialized.

```

# Load original BigVGAN generator weights (24kHz)
orig_weights_path = "bigvgan_generator.pth" # path to Model A's weights
state_24k = torch.load(orig_weights_path, map_location="cpu")
if isinstance(state_24k, dict) and "state_dict" in state_24k:
    state_24k = state_24k["state_dict"] # handle checkpoint vs state_dict

# Prepare new config as AttrDict
h = AttrDict({
    "upsample_rates": new_upsample_rates,

```

```

"upsample_kernel_sizes": new_upsample_kernels,
"upsample_initial_channel": 1536,           # same as original
"resblock": "1",
"resblock_kernel_sizes": [3, 7, 11],       # same patterns as original 14
"resblock_dilation_sizes": [[1,3,5]] * 3,  # three sets of dilations
"gpt_dim": 1024,                           # GPT acoustic feature dim (unchanged)
"activation": "snakebeta",
"snake_logscale": True,
"feat_upsample": False,
"speaker_embedding_dim": 512,
"cond_d_vector_in_each_upsampling_layer": True,
"num_mels": 100,
})
# Instantiate new 44.1 kHz model
model_44k = BigVGAN_Generator(h)
model_44k.eval() # set eval mode (not strictly necessary for weight loading)

```

The `state_24k` dictionary contains the pretrained weights from the 24 kHz model. We construct the new model with the updated config (44 kHz). At this point, layers like the first upsampling `ConvTranspose1d` (`ups.0.0`) have different shapes (e.g. kernel size 16 vs 8, stride 8 vs 4) compared to the old model, so their weights cannot be directly copied.

4. Transfer Weights and Initialize New Layers

We copy matching weights from the old state dict to the new model. For layers whose shape has changed (mismatch in dimensions), we retain the new model's randomly initialized weights and will report them.

```

new_state = model_44k.state_dict()          # new model state dict
converted_state = {}                        # will fill this with combined weights

mismatched = []                            # track keys that cannot be loaded
for name, weight in state_24k.items():
    if name in new_state:
        if weight.size() == new_state[name].size():
            converted_state[name] = weight # shape matches: copy weight
        else:
            # shape mismatch: skip copying this weight
            mismatched.append(name)
            # Also skip associated weight_g if weight_v (for weight normalization)
            if name.endswith("weight_v"):
                g_key = name[:-8] + "weight_g"
                if g_key in new_state:
                    mismatched.append(g_key)
# For any keys not present in old weights (newly added layers), keep initial random values
for name, weight in new_state.items():
    if name not in converted_state:
        converted_state[name] = weight     # use already-initialized weight

```

```
# Load the combined state dict into the new model
model_44k.load_state_dict(converted_state)
```

We iterate through each parameter in the `original_state`: - If the parameter exists in the new model with the same shape, we copy it. - If not (e.g. `ups.0.0.weight_v` for the first upsampling conv, which changed from kernel 8→16), we skip it and record the name. For weight-normalized layers, we also skip the corresponding `weight_g` to avoid inconsistent scaling.

All parameters not found in the old model (e.g. any new layers) remain at their random initialization in `new_state`. After constructing the `converted_state`, we load it into the new model. Using `strict=False` or catching exceptions would also handle missing keys, but here we explicitly control which weights are copied.

5. Report and Validate the Conversion

We print out which layers were newly initialized due to shape mismatch. These should correspond to the upsampling filters whose shapes changed (and any related weightnorm parameters). We then verify that the new model has loaded all other weights and produce a new checkpoint file.

```
# Print mismatched (initialized) parameters for transparency
for key in sorted(set(mismatched)):
    print(f"Initialized new parameter (no matching weight): {key}")

# (Optional) Validate that all expected keys are loaded
own_keys = set(new_state.keys())
loaded_keys = set(converted_state.keys())
missing = own_keys - loaded_keys
unexpected = loaded_keys - own_keys
print(f"Missing keys after loading: {missing}")
print(f"Unexpected keys (should be none): {unexpected}")

# Save the adapted model weights
torch.save(model_44k.state_dict(), "bigvgan_generator_44k.pth")
print("Converted model weights saved to bigvgan_generator_44k.pth")
```

Running this script will output the names of any parameters that were not found or had size mismatches and were therefore re-initialized. For example, you should see the first upsampling layer's weights listed (e.g. `"ups.0.0.weight_v"` and `"ups.0.0.weight_g"`), since we changed its kernel size from 8 to 16. All other layers (including remaining upsampling layers, ResBlocks, speaker encoder, etc.) should load from the pretrained weights (their shapes remain unchanged between 24k and 44k configurations).

Finally, the new generator state dict is saved as `bigvgan_generator_44k.pth`, ready for use in the 44 kHz IndexTTS model. The model architecture is structurally adapted to 44.1 kHz, and any new parameters are initialized (you may fine-tune these on data if needed to fully match performance).

3 4 8 9 config.yaml • IndexTeam/Index-TTS at refs/pr/6

<https://huggingface.co/IndexTeam/Index-TTS/blob/refs%2Fpr%2F6/config.yaml>

5 indextts/BigVGAN/activations.py • IndexTeam/IndexTTS at main

<https://huggingface.co/spaces/IndexTeam/IndexTTS/blob/main/indextts/BigVGAN/activations.py>

6 7 10 11 12 13 14 indextts/BigVGAN/models.py • IndexTeam/IndexTTS at main

<https://huggingface.co/spaces/IndexTeam/IndexTTS/blob/main/indextts/BigVGAN/models.py>